Operations Research II

Optimization in “ Cafe ” Operations

2020170822 권민석
2020170803 유원준
2022170829 박소영
2022170844 조혜린

Table of Contents

01 연구 배경

02 Task 1

03 Task 2

04 한계 및 제언

01 연구 배경 - Task 1

카페 내 **최적의**

2인, 4인 좌석

비율 결정

좌석 활용도 증가

고객 만족도 향상

카페 운영 효율성 증가

01 연구 배경 - Task 2

카페 상황별

최적의 Policy

조합 도출

수익성 증대

고객 만족도 향상

상황별 유연한 대처 가능

TASK 1 . 카페 내 최적의 2인, 4인 좌석 비율 결정

02 Task 1 - 데이터 수집 과정

1. 입장 시간 데이터

고려대학교 학생들이 공부를 주 목적으로 사용하는 **'Cafe Ann'** 에서 팀원들이 24시간 관찰을 통하여 수집한 고객 데이터이다.

시간	인원
1:29	1
1:43	1
2:49	1
3:11	1
3:44	2
7:01	2
8:56	1
9:05	1
9:24	1
10:44	1
10:47	1
10:56	1
11:10	1
11:31	1
13:47	2

카페 ANN 고객 출입 Data (2025.05.15 기준)

수집 과정

24시간 동안 사람이 출입하여 결제하였을 때

인원 수 & 시간대 정보 기록

λ (총합)= 7.17

λ (1인) = 5.33 ✓

λ (2인 ↑)= 1.84 ✓

02 Task 1 - 데이터 수집 과정

2. 카페 매출 데이터

판매일	판매건수	매출액	건당 매출액
20240415	202	1113100	5510
20240416	241	1105500	4587
20240417	222	1179600	5314
20240418	239	1168700	4890
20240419	187	1058300	5659
20240420	211	1002400	4751
20240421	244	1270800	5208
20240422	237	1237000	5219
20240423	203	1154800	5689
20240424	226	1157700	5123
20240425	206	1143500	5551
20240426	182	995400	5469
			평균
			5247.5

'cafe Ann' 의 매출 내역

수집 과정

: 해당 카페를 사용하는 **인원 당 예상 이익**

(단, 24시간 대기해본 결과 TAKE OUT으로

구매하는 사람은 극 소수로 관측하였기에

모든 결제는 카페 이용으로 이어진다고 가정)

02 Task 1 - 데이터 수집 과정

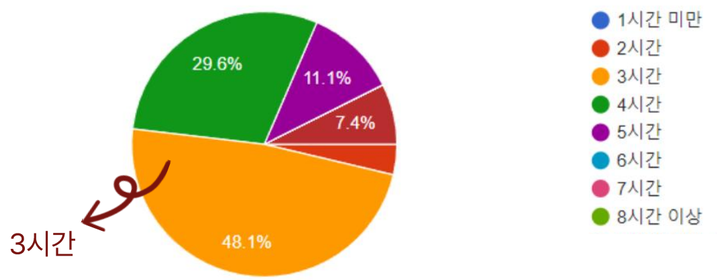
3. 사용 시간 데이터

카페 공부(카공)을 목적으로 카페를 이용하는

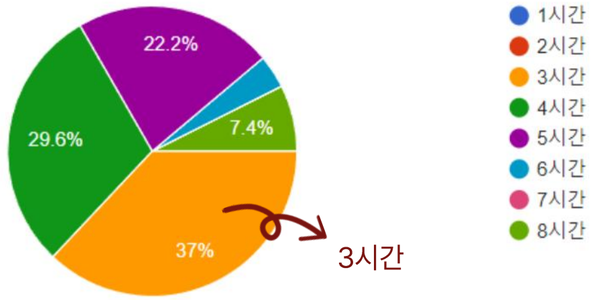
대학생을 대상으로 **카페 사용시간 설문조사** 진행

공부 목적으로 카페 이용시 **평균 이용 시간** 계산

평소 공부를 목적으로 카페를 이용하실 때, 평균 몇 시간 정도 이용하시나요? (대략적으로)



만약 카페에 '이용 시간 제한'을 둔다고 몇 시간 정도가 적절할까요?



02 Task 1 - 모델링

1. 시스템 정의

System : 카페

사람이 카페에 입장하는 rate = **arrival rate**(λ)

카페 사용을 마치고 나오는 rate = **service rate**(μ)

카페의 좌석 수 = **server** (s)

= 카페의 수용 인원 **K** (∵ 카페가 FULL 인 경우 고객은 돌아감 = queue 존재 X)

∴ 해당 카페는 **M/M/K/K Queueing Model** 사용

02 Task 1 - 모델링

2. Markov properties에 대한 검정

μ : 설문을 통해 얻은 사용시간 평균 -> **지수분포**를 따른다고 가정

λ : 직접 구한 데이터 ->**지수분포**를 따르는지 여부 검증 필요

=>이에 대한 히스토그램 + 확률밀도함수(PDF)를 중첩하여

시각화 & Kolmogorov-Smirnov (K-S) 검정을 진행

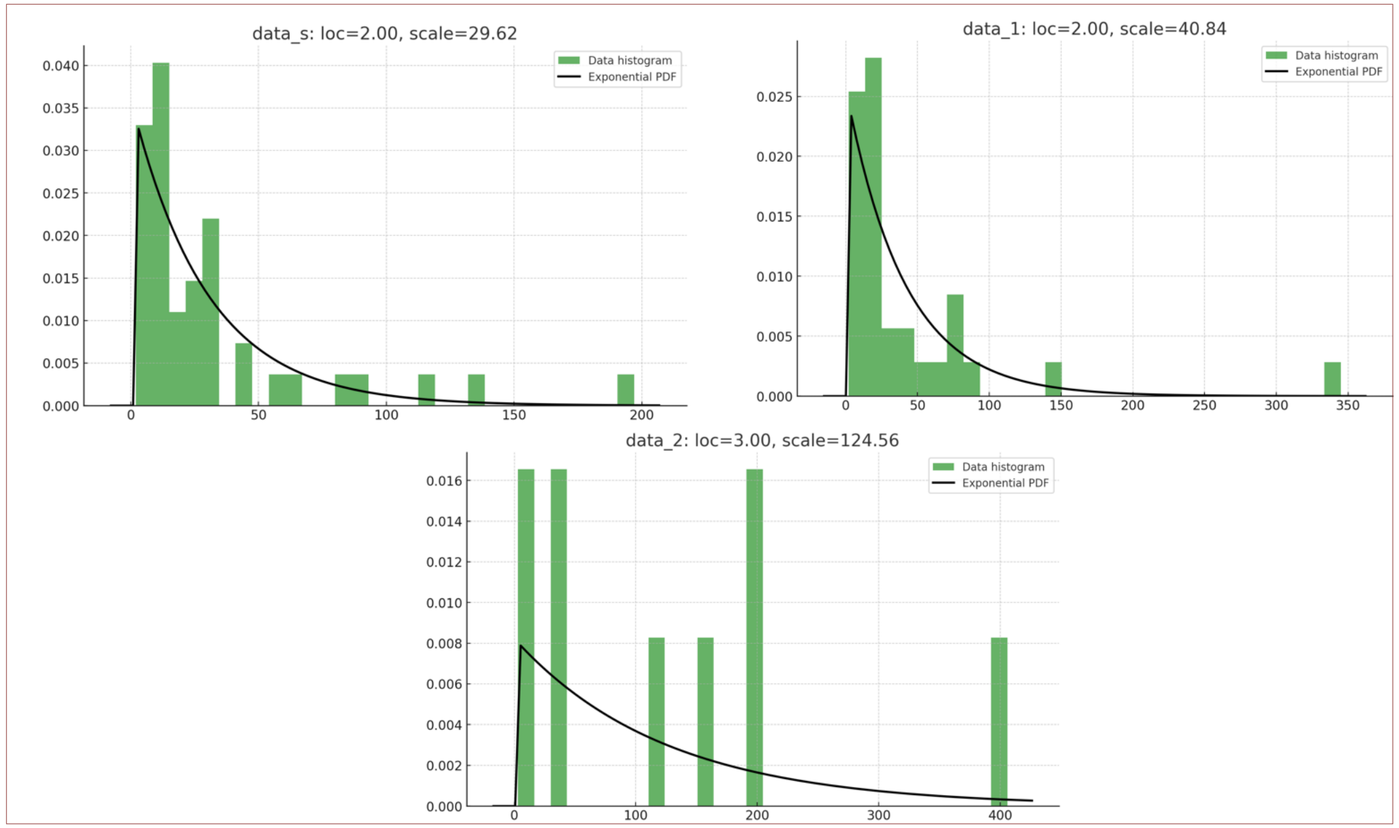
data_s : 모든 방문객에 대한 시간 간격
data_1 : 1인 방문객에 대한 시간 간격
data_2 : 2인 이상 방문객에 대한 시간 간격

data_s	data_1	data_2
14	14	197
66	66	406
22	22	161
33	345	195
197	9	3
115	19	4
9	80	30

↳ **방문시간 간격** 에 대한 데이터

02 Task 1 - 모델링

2. Markov properties에 대한 검정



<검정 결과>

H_0 : 데이터가 지수분포를 따른다

H_1 : 데이터가 지수분포를 따르지 않는다

	Kolmogorov-Smirnov (K-S) 검정	p-value
data_s	0.14307078613201918	0.32475999384319
data_1	0.19640687029868398	0.15970929471442163
data_2	0.23673441297449485	0.6138049845540922

p-value >0.05 (유의수준)

∴ 모든 데이터의 귀무가설을 기각할 수 X

=> 데이터가 **지수분포**를 따른다

02 Task 1 - 모델링

3. 변수 설정 : 수집한 데이터를 통해서 해당 모델에 사용할 parameter값을 정의

λ = 43/6

→ (from 입장 시간 데이터) 24시간 동안 입장한 횟수 172 ÷ 24시간 => 43/6

μ = 1/3.48

→ (from 사용 시간 데이터) 사용 시간 평균 계산 값 = 111.38 ÷ 32명 => 3.48

s = 34

→ 카페에 1인당 2인테이블 하나를 사용한다 가정 = 2인 테이블 개수 (Cafe ANN 기준)

profit = 5300

→ (from 카페 매출 데이터) 수익 총합 ÷ 결제수 총합 = 5247.5 , 반올림한 값

02 Task 1 - 모델링

4. M/M/K/K 모델에 대한 평가지표

: 해당 모델은 M/M/S/K 모델에서 S=K 인 Erlang Loss system (특수 case)

- M/M/S/K model의 지표 특수화 과정

$$*P_n = \frac{(\lambda/\mu)^n}{n!} P_0 \quad (0 \le n < s)$$

$$P_n = \frac{(\lambda/\mu)^n}{s^{n-s} s!} P_0 \quad (s \le n \le K)$$

$$P_0 = \left(\sum_{n=0}^{s-1} \frac{(\lambda/\mu)^n}{n!} + \sum_{n=s}^{K} \frac{(\lambda/\mu)^n}{s^{n-s} s!} \right)^{-1}$$

(before)

$$P_0 = \left(\sum_{n=0}^{s} \frac{(\lambda/\mu)^n}{n!} \right)^{-1}$$

$$P_s = \frac{(\lambda/\mu)^s}{s!} P_0$$

$$P_s = \frac{(\lambda/\mu)^s}{s!} \left(\sum_{n=0}^{s} \frac{(\lambda/\mu)^n}{n!} \right)^{-1}$$

(after)

∵ **Erlang-B model (K=S)**

-> system에 존재하는 인원수가 0 ≤ n ≤ S (서버 수 = 시스템 용량)

* Pn = 시스템에 n명이 있을 확률

02 Task 1 - 모델링

4. M/M/K/K 모델에 대한 평가지표

$$P_s = \frac{(\lambda/\mu)^s}{s!}\left(\sum_{n=0}^{s}\frac{(\lambda/\mu)^n}{n!}\right)^{-1}$$

-> system에 s명이 차있는 상태 = 카페가 FULL 상태일 확률

= 도착하는 인원에 대한 Barking Probability

$$B(E,m) = \left(\frac{\frac{E^m}{m!}}{\sum_{k=0}^{m}\frac{E^k}{k!}}\right)$$

* E=λ/μ 와 m=server 입력

1인당 profit을 고려 -> 해당 queueing model의 시간당 이익 (revenue per hour) 계산

- 카페를 방문하였을 때 서비스를 받는 고객의 비율 : 1 – B(E , m)

- 단위 시간당 서비스를 받는 고객 수 : λ * (1 – B(E , m))

- 단위 시간 당 서비스 이익 : profit * λ * (1 – B(E , m))

∴ λ, μ, s, profit 정보를 사용하여 **카페의 1시간 매출** 예상 가능

02 Task 1 - 모델링

4. M/M/K/K 모델에 대한 평가지표

+) simulation 과정, python을 통해 함수를 정의하고 값을 구함

```python
import numpy as np
from scipy.stats import expon
from scipy.integrate import quad
import matplotlib.pyplot as plt
import math

#초기 데이터 설정
lambda_rate =  43/6    # Arrival rate
mu_rate = 1/3.48    # Service rate
cost = 5300   # revenue_per_customer
servers = 34 # 카페 좌석수(s = k = 34)

# mmkk model fomula
def mmkk_model(traffic_intensity, servers):

    numerator = (traffic_intensity ** servers) / math.factorial(servers)
    denominator = sum((traffic_intensity ** k) / math.factorial(k) for k in range(servers + 1))
    return numerator / denominator

# 이익 계산
def calculate_revenue(lambda_rate, mu_rate, servers, revenue_per_customer):

    traffic_intensity = lambda_rate / mu_rate
    blocking_probability = mmkk_model(traffic_intensity, servers)

    served_customers = lambda_rate * (1 - blocking_probability)
    total_revenue = served_customers * revenue_per_customer * 720

    return {
        'blocking_probability': blocking_probability,
        'served_customers': served_customers,
        'total_revenue': total_revenue
    }

# 현재 변수를 사용하여 계산
results = calculate_revenue(lambda_rate, mu_rate, servers, cost)
print(f"Blocking Probability: {results['blocking_probability']:.4f}")
print(f"Served Customers per hour: {results['served_customers']:.2f}")
print(f"Total Revenue per month: ${results['total_revenue']:.2f}")
```

```
Blocking Probability: 0.0161
Served Customers per hour: 7.05
Total Revenue per month: 26906975.37
```

1달 기준을 1일 기준으로 변경시 : 26906975.37 / 30 = **896889.779 원**

=> 실제 카페 매출과 유사한 값이 도출

02 Task 1 - 모델링

5. 카페 좌석 최적화

- 카페 가정 상황

<가정1> 공부를 위해 카페를 방문하는 사람은 테이블의 협소한 크기로 인해 **1인이 테이블 1개를 모두 사용**한다

<가정2> 테이블을 2개로 붙일 시 1인 사용 제한이 존재한다 & 2인이상 방문 고객은 2인 테이블만 사용한다

<가정3> 2인 이상이 방문하였을 때는 1인이 방문하였을 때와 다르게 메뉴선택 폭이 늘어남-> 금액에 차등을 둔다

(1인 : 5000원, 2인 이상 : 11000원)

- 기존 좌석배치 : 28개의 1인 좌석 테이블 (테이블 1개 + 의자 2개) & 3개의 2인 좌석 테이블 (테이블 2개 +의자 4개)

=> 1인 방문자와 2인 방문자의 비율은 고려하지 않은 **임의의 값**

02 Task 1 - 모델링

5. 카페 좌석 최적화

목표 : 테이블 수 조정을 통한 수익의 최대화

<가정2> 와 현재 카페 ANN의 상황을 고려했을 때 (1인석) = 34 - 2 * (2인석)

∴ 3 <= (2인석) <= 16

```
# 도착 인원 type별 lambda와 revenue 정의

A_lambda_rate = 16/3  # 1인 Arrival
A_cost = 5000 # 1인  service시 revenue

B_lambda_rate = 5/3  # 2인 이상 Arrival
B_cost = 11000 # 2인 이상 service시 revenue
```

```
# 최적값 (max revenue) 출력

available_revenue =[]
size_table = {}

for k_2 in range (3, 17):

    k_1 = servers - 2 * k_2
    results_1 = calculate_revenue(A_lambda_rate, mu_rate, k_1, A_cost)
    results_2 = calculate_revenue(B_lambda_rate, mu_rate, k_2, B_cost)
    available_revenue.append(results_1['total_revenue'] + results_2['total_revenue'])
    size_table[results_1['total_revenue'] + results_2['total_revenue']] = k_2

opt_size = size_table[max(available_revenue)]
k_1 = servers - 2 * opt_size

print("최적화된 테이블 배치와 기대 수익")
print(f'1인석 {k_1}개, 2인석 {opt_size}개')
print(f"한달 기대 수익: ${max(available_revenue):.2f}")
```

각 Case별로 이전의 모델평가 코드를 반복적으로 이용하여 최적값을 찾는 코드 실행

<결과>

```
최적화된 테이블 배치와 기대 수익
1인석 20개, 2인석 7개
한달 기대 수익: 27770476.26
```

2인석 3개 ->7개로 변화시켜 **2인 이상 고객 수용율↑**

27770476.26 원 > 기존 시스템 (26906975.37원)

∴ 의미 있는 차이가 존재 !

TASK 2 . 카페 상황별 최적의 Policy 조합 도출

03 Task 2 - 문제 정의

1. State 정의

State	ρ	λ	λ(대표)	L
0	$0\% \le \rho < 40\%$	$0 \le \lambda < 4$	**2**	**7명**
1	$40\% \le \rho < 70\%$	$4 \le \lambda < 7$	**5.5**	**19명**
2	$70\% \le \rho < 100\%$	$7 \le \lambda < 10$	**8.5**	**28명**
3	$100\% \le \rho < 140\%$	$10 \le \lambda < 14$	**12**	**31명**
4	$140\% \le \rho < 200\%$	$14 \le \lambda < 20$	**17**	**33명**

State : 한 달간 카페 방문률

주요 가정

카페앤과 같은 조건의 24시간 카페

S(number of Servers) = 34

μ (Rate of Service) = 1/3.4 ≈ 0.29

포화도(ρ) = λ/sμ

Ex) 카페 앤 λ=43/6 -> ρ = 71.6% -> state 2

03 Task 2 - 문제 정의

2. Decision k

Decision	Action	Relevant States
1	**Do Nothing**	**0, 1, 2, 3, 4**
2	**쿠키 서비스**	**0, 1**
3	**이용 시간 제한**	**3, 4**

Decision 설명

- **k = 1 : Do Nothing, 모든 state**에서 가능
- **k = 2 :** 고객 유지/증가시키기 위한 무료 쿠키 서비스
- **k = 3 : 3시간 이용 제한**으로 **balking** 비율 ↓

=> 순환율을 높이는 전략

03 Task 2 - 문제 정의

※ Decision 3 설명

Service time = 3에서 끊긴 새로운 μ의 지수분포 설정

$$f_{\text{trunc}}(t|\lambda <= t_{\max}) = \begin{cases} \frac{\lambda e^{-\lambda t}}{1-e^{-\lambda t_{\max}}} & \text{if } 0 \le t \le t_{\max} \\ 0 & \text{otherwise} \end{cases}$$

```
# 절단된 지수분포의 pdf 정의
def truncated_expon_pdf(t, lambda_, max_time):
    normalization_constant = 1 / (1 - np.exp(-lambda_ * max_time))
    return lambda_ * np.exp(-lambda_ * t) * normalization_constant
```

절단된 지수분포의 확률밀도함수 f_trunc

$$\mathbb{E}[T_{\text{trunc}}] = \int_0^{t_{\max}} t f_{\text{trunc}}(t|\lambda <= t_{\max})\, dt$$

$$\mathbb{E}[T_{\text{trunc}}] = \frac{1}{\lambda} - \frac{t_{\max} e^{-\lambda t_{\max}}}{1-e^{-\lambda t_{\max}}}$$

```
# 절단된 지수분포의 평균 서비스 시간 계산
def expected_truncated_expon(lambda_, max_time):
    integral, _ = quad(lambda t: t * truncated_expon_pdf(t, lambda_, max_time), 0, max_time)
    return integral

def make_exp_distribution(lambda_, max_time):
    # 평균 서비스 시간과 서비스율 계산
    mean_service_time = expected_truncated_expon(lambda_, max_time)
    new_mu_rate = 1 / mean_service_time

    print(f"Calculated mean service time for truncated exponential distribution: {mean_service_time:.4f} hours")
    print(f"Calculated service rate (mu) for truncated exponential distribution: {new_mu_rate:.4f} per hour")
```

$$\mu = \frac{1}{\mathbb{E}[T_{\text{trunc}}]}$$

서비스율 **μ = 1/(평균 서비스 시간)**

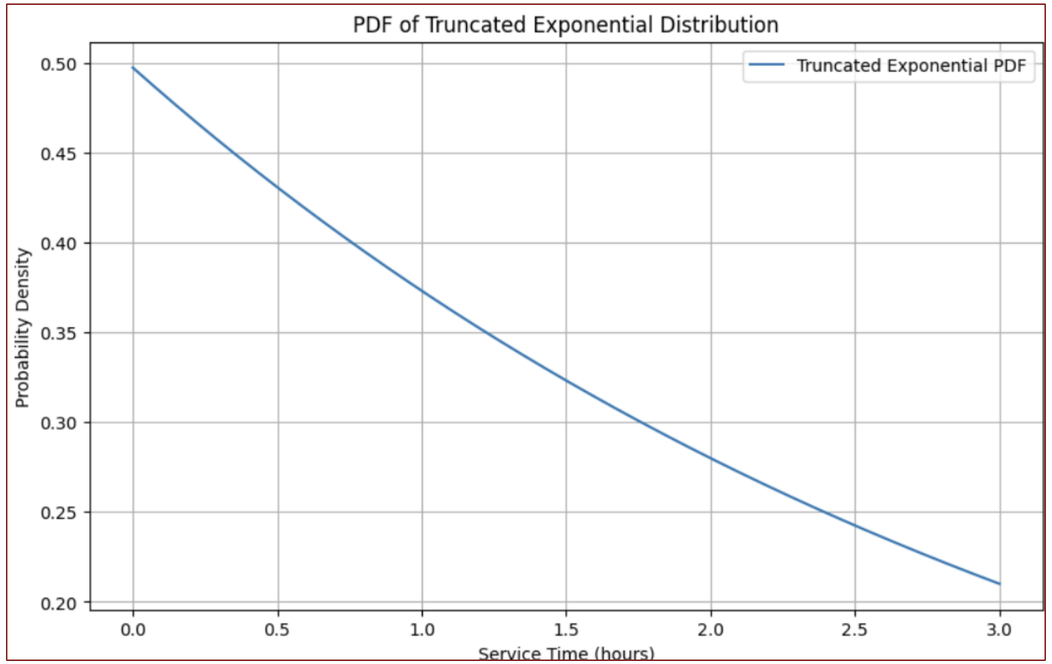
03 Task 2 - 문제 정의

※ Decision 3 설명

Service time = 3에서 끊긴 새로운 μ의 지수분포 설정

```python
def make_exp_distribution(lambda_, max_time):
    # 평균 서비스 시간과 서비스율 계산
    mean_service_time = expected_truncated_expon(lambda_, max_time)
    new_mu_rate = 1 / mean_service_time

    print(f"Calculated mean service time for truncated exponential distribution: {mean_service_time:.4f} hours")
    print(f"Calculated service rate (mu) for truncated exponential distribution: {new_mu_rate:.4f} per hour")

    # 서비스 시간 분포 시각화
    service_times = np.linspace(0, max_time, 1000)
    pdf_values = truncated_expon_pdf(service_times, lambda_, max_time)

    plt.figure(figsize=(10, 6))
    plt.plot(service_times, pdf_values, label='Truncated Exponential PDF')
    plt.xlabel('Service Time (hours)')
    plt.ylabel('Probability Density')
    plt.title('PDF of Truncated Exponential Distribution')
    plt.legend()
    plt.grid(True)
    plt.show()

# 3시간 제한에 대한 서비스 지수분포
lambda_ = mu_rate  # 원래 service rate 지수 분포의 lambda (평균 4시간)
max_time = 3  # 절단 시간 <----변경해보기

make_exp_distribution(lambda_, max_time)
```

t_max = 3 : 3시간 제한



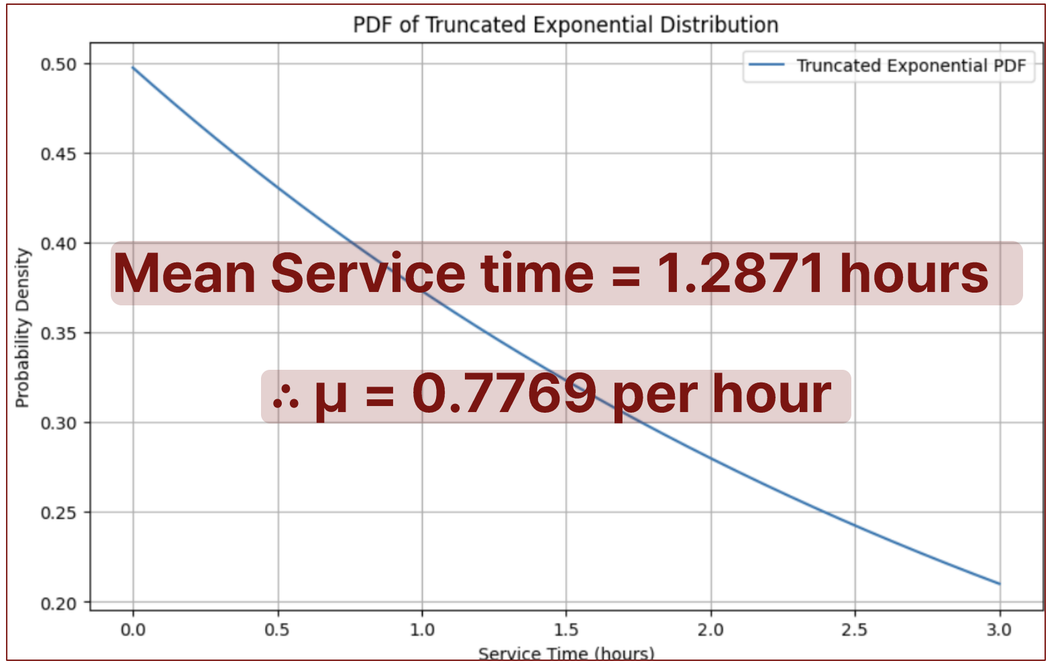
03 Task 2 - 문제 정의

※ Decision 3 설명

Service time = 3에서 끊긴 새로운 μ의 지수분포 설정

```python
def make_exp_distribution(lambda_, max_time):
    # 평균 서비스 시간과 서비스율 계산
    mean_service_time = expected_truncated_expon(lambda_, max_time)
    new_mu_rate = 1 / mean_service_time

    print(f"Calculated mean service time for truncated exponential distribution: {mean_service_time:.4f} hours")
    print(f"Calculated service rate (mu) for truncated exponential distribution: {new_mu_rate:.4f} per hour")

    # 서비스 시간 분포 시각화
    service_times = np.linspace(0, max_time, 1000)
    pdf_values = truncated_expon_pdf(service_times, lambda_, max_time)

    plt.figure(figsize=(10, 6))
    plt.plot(service_times, pdf_values, label='Truncated Exponential PDF')
    plt.xlabel('Service Time (hours)')
    plt.ylabel('Probability Density')
    plt.title('PDF of Truncated Exponential Distribution')
    plt.legend()
    plt.grid(True)
    plt.show()

# 3시간 제한에 대한 서비스 지수분포
lambda_ = mu_rate  # 원래 service rate 지수 분포의 lambda (평균 4시간)
max_time = 3  # 절단 시간 <----변경해보기

make_exp_distribution(lambda_, max_time)
```

t_max = 3 : 3시간 제한



Mean Service time = 1.2871 hours

∴ μ = 0.7769 per hour

03 Task 2 - 문제 정의

3. Probability Pij(k)

k = 1 : Do Nothing

i / j	0	1	2	3	4
0	**3/4**	**1/8**	**1/16**	**3/64**	**1/64**
1	**1/4**	**1/2**	**1/8**	**1/16**	**1/16**
2	**1/16**	**3/16**	**1/2**	**3/16**	**1/16**
3	**1/16**	**1/16**	**1/8**	**1/2**	**1/4**
4	**1/64**	**3/64**	**1/16**	**1/8**	**3/4**

확률 설정 과정

: State 넘어갈수록 **확률이 적어지도록** 임의로 설정

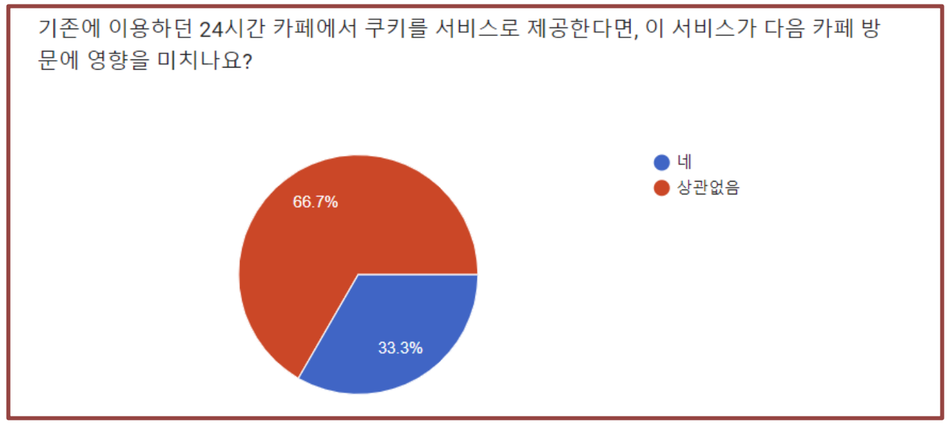
03 Task 2 - 문제 정의

3. Probability Pij(k)

k = 2 : 쿠키 서비스

i / j	0	1	2	3	4
0	**2/3**	**1/3**	**0**	**0**	**0**
1	**0**	**2/3**	**1/3**	**0**	**0**
2	-	-	-	-	-
3	-	-	-	-	-
4	-	-	-	-	-

확률 설정 과정



Pii(2) = 2/3

Pii+1(2) = 1/3

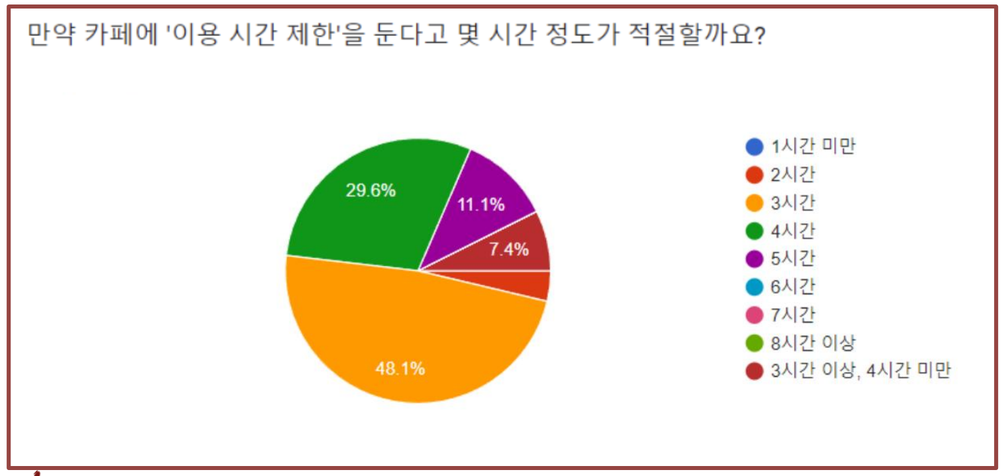
03 Task 2 - 문제 정의

3. Probability Pij(k)

k = 3 : 이용 시간 제한

i / j	0	1	2	3	4
0	-	-	-	-	-
1	-	-	-	-	-
2	-	-	-	-	-
3	**0**	**2/5**	**3/5**	**0**	**0**
4	**0**	**0**	**1/2**	**1/2**	**0**

확률 설정 과정



3 시간 이용 제한 시 기존 고객의 40% 방문 X

∴ state 3&4의 각 λ 범위에 60% 를 곱함

03 Task 2 - 문제 정의

3. Probability Pij(k)

k = 3 : 이용 시간 제한

i / j	0	1	2	3	4
0	-	-	-	-	-
1	-	-	-	-	-
2	-	-	-	-	-
3	**0**	**2/5**	**3/5**	**0**	**0**
4	**0**	**0**	**1/2**	**1/2**	**0**

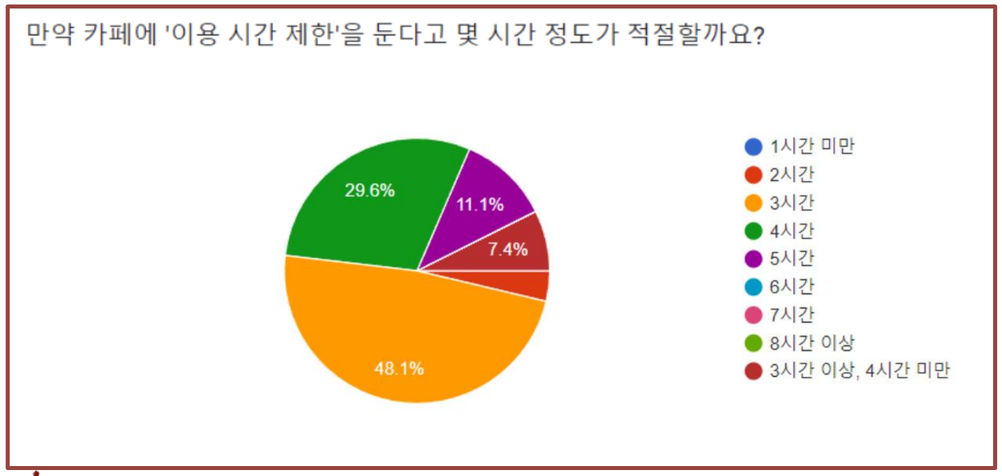
1	$40\% \le \rho < 70\%$	$4 \le \lambda < 7$
2	$70\% \le \rho < 100\%$	$7 \le \lambda < 10$

*State 정의

state 3 : $10 \le \lambda < 14$

$\Rightarrow 6 \le 0.6\lambda < 8.4$

확률 설정 과정



3 시간 이용 제한 시 기존 고객의 40% 방문 X

∴ state 3&4의 각 λ 범위에 60% 를 곱함

03 Task 2 - 문제 정의

3. Probability Pij(k)

k = 3 : 이용 시간 제한

i / j	0	1	2	3	4
0	-	-	-	-	-
1	-	-	-	-	-
2	-	-	-	-	-
3	**0**	**2/5**	**3/5**	**0**	**0**
4	**0**	**0**	**1/2**	**1/2**	**0**

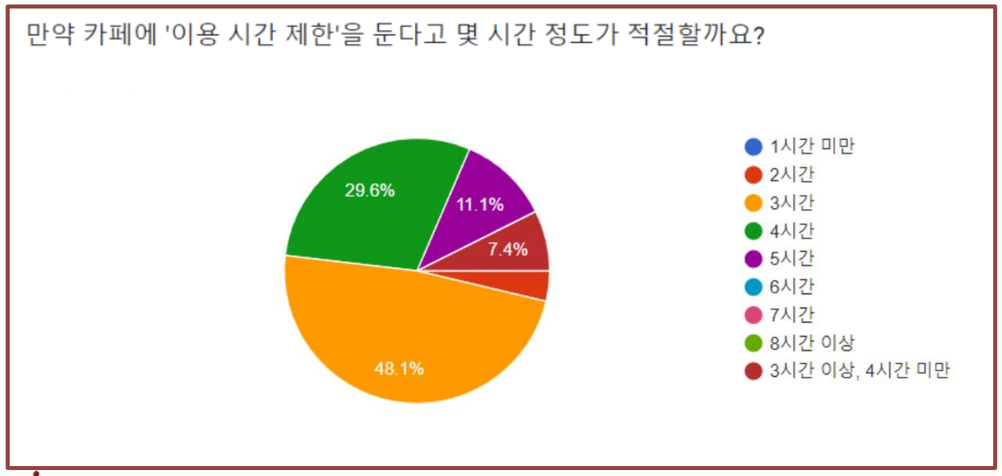
2	$70\% \le \rho < 100\%$	$7 \le \lambda < 10$
3	$100\% \le \rho < 140\%$	$10 \le \lambda < 14$

*State 정의

state 4 : 14 ≤ λ < 20

⇒ 8.4 ≤ 0.6λ < 12

확률 설정 과정



3 시간 이용 제한 시 기존 고객의 40% 방문 X

∴ state 3&4의 각 λ 범위에 60% 를 곱함

03 Task 2 - 문제 정의

4. Net Profit NPik

k / i	0	1	2	3	4
1	₩7,632	₩20,988	₩32,436	₩38,160	₩38,160
2	₩7,200	₩20,232	0	0	0
3	0	0	0	₩45,792	₩64,872

NPik 설정 과정

- NPi1

= 고객 수/hour * 24(hours) * 30(days)

* 5,300(인 당 평균 주문 비용)

※ **고객 수/hour** = min(λ, sμ)

03 Task 2 - 문제 정의

4. Net Profit NPik

k / i	0	1	2	3	4
1	₩7,632	₩20,988	₩32,436	₩38,160	₩38,160
2	₩7,200	₩20,232	0	0	0
3	0	0	0	₩45,792	₩64,872

NPik 설정 과정

- NPi2

= NPi1 - 고정비(쿠키 주문 비용)

- 쿠키 = ₩150 / 개

⇒ state 0 고정비 = 150 * 4 * 24 * 30 = ₩432,000

⇒ state 1 고정비 = 150 * 7 * 24 * 30 = ₩756,000

시간 당 최대 예상 고객 수

03 Task 2 - 문제 정의

4. Net Profit NPik

k / i	0	1	2	3	4
1	₩7,632	₩20,988	₩32,436	₩38,160	₩38,160
2	₩7,200	₩20,232	0	0	0
3	0	0	0	₩45,792	₩64,872

NPik 설정 과정

- NPi3

= 고객 수/hour * 24(hours) * 30(days)

* 5,300(인 당 평균 주문 비용)

- **고객 수/hour =** min(λ, sμ)

⇒ NPi1과 차이점 : μ = 0.77로 증가 ∴ 고객수/hour ⇒ λ 채택

03 Task 2 - 모델링

1. A Linear Programming Model

※Decision Variable

$y_{ik} = P\{\text{state} = i \text{ and decision} = k\}$

For each $i = 0, 1, \ldots, M$ and $k = 1, 2, \ldots, K$

$$\textit{Maximize} \qquad Z = \sum_{i=0}^{M}\sum_{k=1}^{K} NP_{ik}y_{ik}$$

subject to the constraints

$$(1) \qquad \sum_{i=0}^{M}\sum_{k=1}^{K} y_{ik} = 1.$$

$$(2) \qquad \sum_{k=1}^{K} y_{jk} - \sum_{i=0}^{M}\sum_{k=1}^{K} y_{ik}p_{ij}(k) = 0, \qquad \text{for } j = 0, 1, \ldots, M.$$

$$(3) \qquad y_{ik} \geq 0, \qquad \text{for } i = 0, 1, \ldots, M;\ k = 1, 2, \ldots, K.$$

03 Task 2 - 모델링

2. Solving by Excel Solver

Decision Variable	y_01	y_02	y_11	y_12	y_21	y_31	y_33	y_41	y_43		
NP	7632	7200	20988	20232	32436	38160	45792	38160	64872		
Objective Max	NP										
Constraint 1	1	1	1	1	1	1	1	1	1	=	1
Constraint 2	0.25	0.333333	-0.25		- 1/16	- 1/16		- 1/64		=	0
Constraint 3	- 1/8	- 1/3	0.5	0.333333	-0.1875	- 1/16	-0.4	- 3/64		=	0
Constraint 4	- 1/16		- 1/8	- 1/3	0.5	- 1/8	-0.6	- 1/16	-0.5	=	0
Constraint 5	- 3/64		- 1/16		- 3/16	0.5	1	- 1/8	-0.5	=	0

03 Task 2 - 모델링

2. Solving by Excel Solver

Decision Variable		y_01	y_02	y_11	y_12	y_21	y_31	y_33	y_41	y_43		
		0.13163	0	0	0.26824	0.3204157	0.206105218	0	0	0.073609006		
NP		7632	7200	20988	20232	32436	38160	45792	38160	64872		
Objective	Max	NP	29464.77									
Constraint 1		1	1	1	1	1	1	1	1	1	=	1
Constraint 2		0.25	0.3333333	-0.25		- 1/16	- 1/16		- 1/64		=	0
Constraint 3		- 1/8	- 1/3	0.5	0.333333	-0.1875	- 1/16	-0.4	- 3/64		=	0
Constraint 4		- 1/16		- 1/8	- 1/3	0.5	- 1/8	-0.6	- 1/16	-0.5	=	0
Constraint 5		- 3/64		- 1/16		- 3/16	0.5	1	- 1/8	-0.5	=	0

LHS	=	RHS
1	=	1
-6.1E-17	=	0
3.64E-17	=	0
0	=	0
-1.6E-16	=	0

03 Task 2 - 모델링

3. Optimal Policy

Decision Variable	y_01	y_02	y_11	y_12	y_21	y_31	y_33	y_41	y_43
	0.13163	0	0	0.26824	0.3204157	0.206105218	0	0	0.073609006

- state 0, 2, 3 : Do Nothing
- state 1 : 쿠키 서비스
- state 4 : 3시간 이용 제한

Policy : (1, 2, 1, 1, 3)

04 한계 및 제언

1. 데이터 접근성의 한계

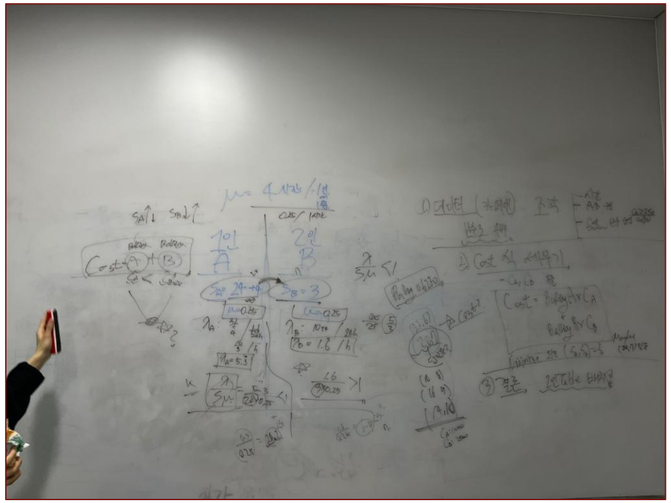
· 확률 설정 by 설문조사 & 가정

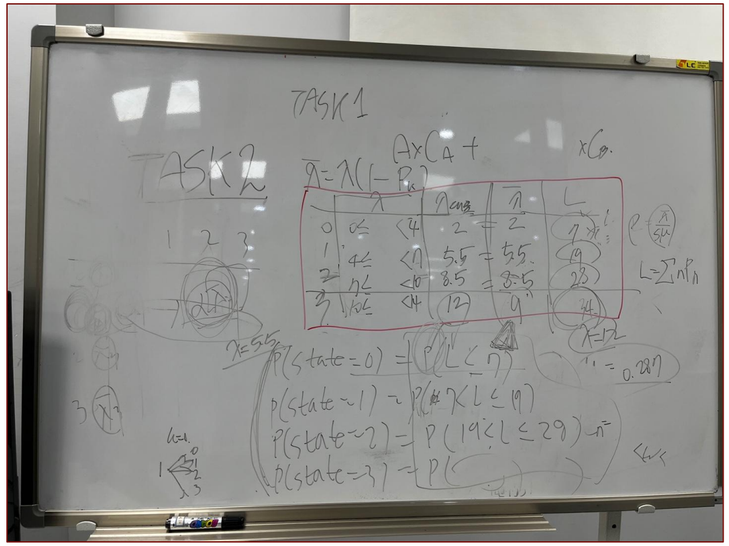
· 고객 당 주문 건 수 부정확

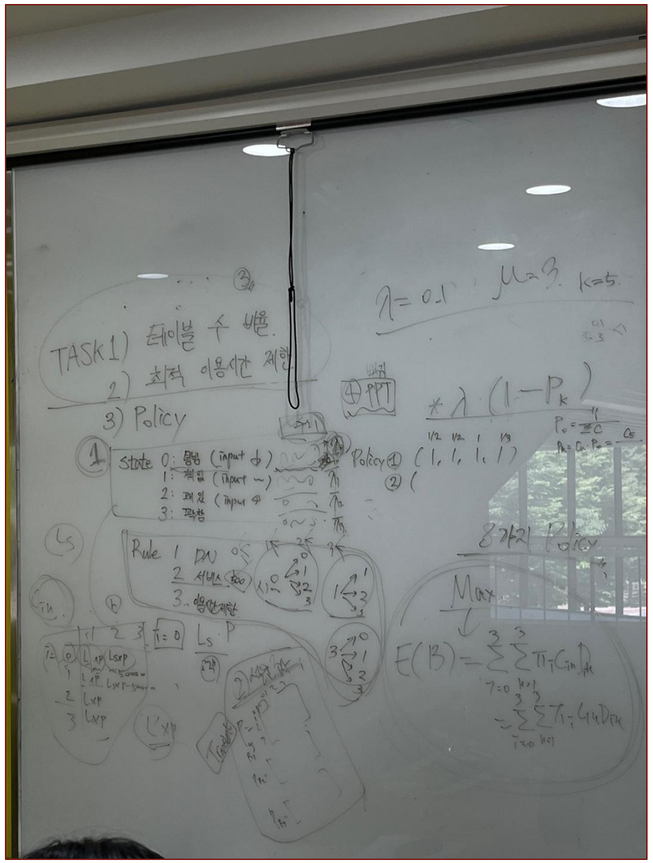
2. Pij(k) 임의 설정

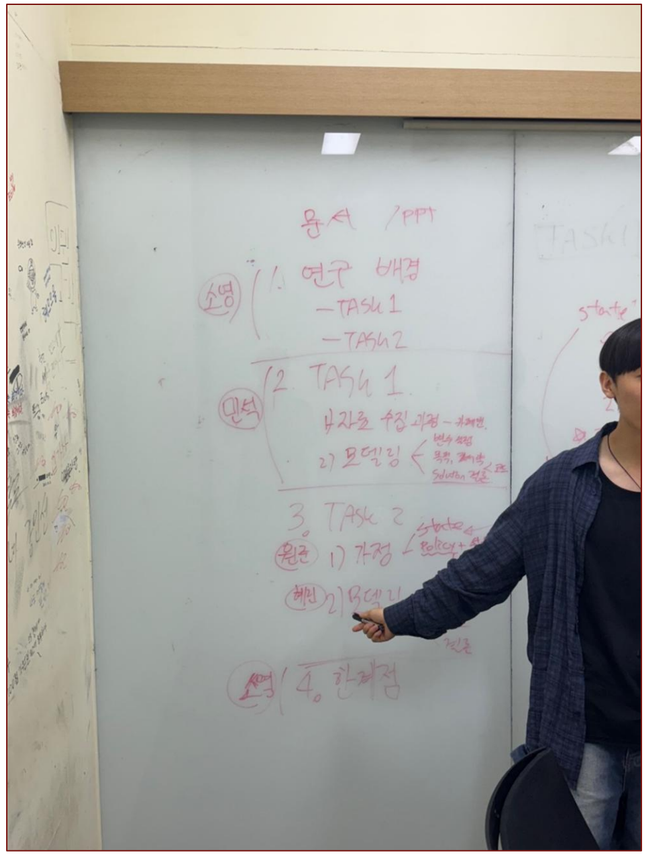
· steady-state probability 존재해야 함

Cookie

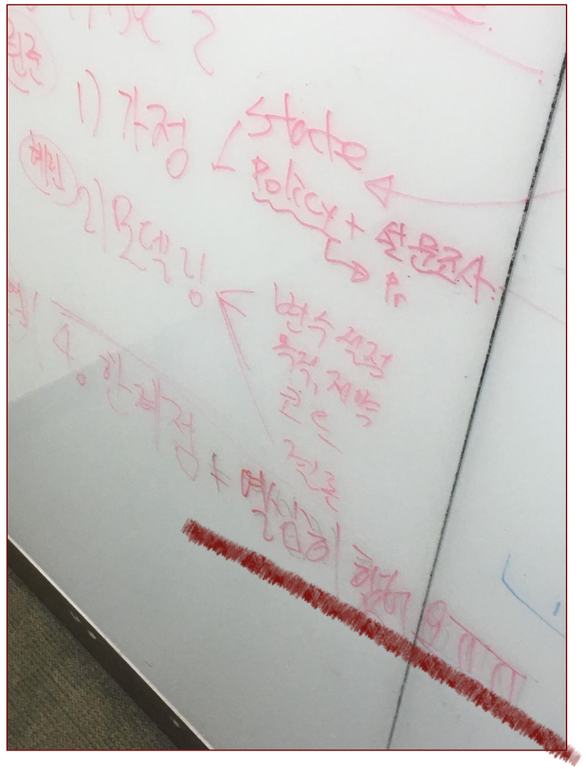
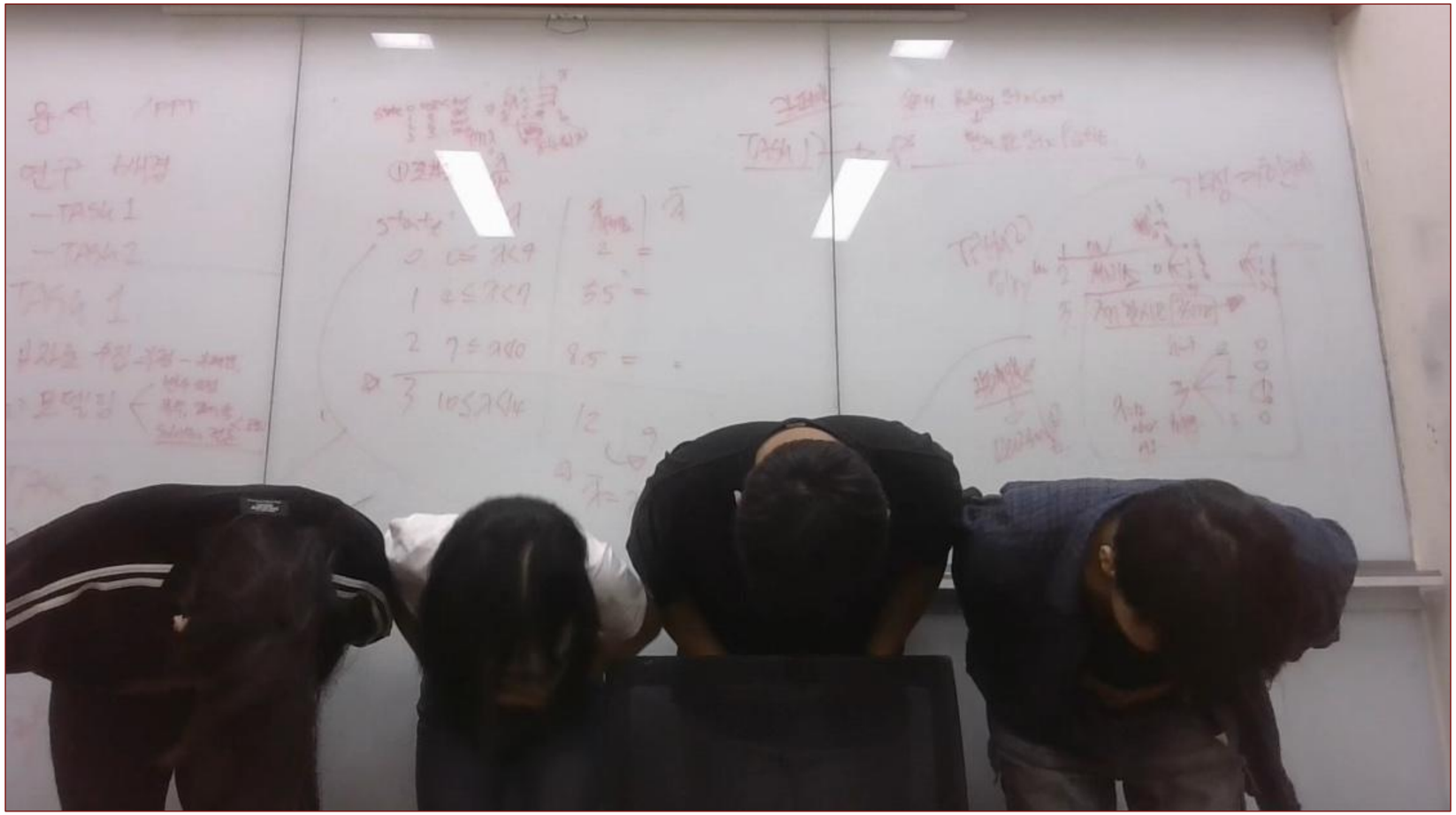



TASK1
TASK2


TASK1) 테이블 수 변화
2) 최적 이용시간 제한
3) Policy


1. 연구 배경
— TASK 1
— TASK 2
2. TASK 1
3. TASK 2
4. 한계점

Cookie



감사합니다